CST2550 - Coursework 2

Project Management

Group Members		
Student ID	**Student Name/Surname**	**Role**
M01015301	Nunkoo Namrata-Riya	Scrum Master
M01038115	Hau Yam Seung Mélanie	Secretary
M01014266	Coret Mégane	Developer 1
M01026956	Ramtohul Syrielle	Developer 2
M01015305	Sedoyeka Rayan	Tester

Submission Date: 15/04/26

Lab Tutor: Dr. Gina Soopramanien

Table of Contents

Table of Figures 4

Table of Tables 4

1. Introduction 5

1.1 Project Description 5

1.2 Problem Addressed 5

1.3 Main Functionalities 5

1.4 Project Objectives 6

2. Team Roles 6

3. Tools 7

3.1 Communication 7

3.2 Documentation 7

3.3 Design 7

3.4 Development Environments 7

3.5 Testing Frameworks 7

3.6 Version Control Management 7

4. Meetings 7

4.1 Frequency of Sprint Meetings 8

4.2 Daily Stand-ups 8

4.3 Recording of Meetings Notes 8

4.4 Documentation of Decisions 8

5. Workload division 8

5.1 Task Distribution 8

5.2 Progress Monitoring 9

5.3 Git Branching Strategy 9

5.4 Integration and Merging 9

6. Project planning 9

6.1 Work Breakdown Structure (WBS) 9

6.2 Gantt Chart 10

7. Project Design and Implementation 10
7.1 Design Decision Coordination 10
7.2 Use of Diagrams in Development 10
7.3 Task Allocation in Implementation 11
7.4 Progress Tracking 11
8. Testing 11
8.1 Bug Reporting Process 11
8.2 Testing Approach 11
8.3 Alignment with Development Sprints 12
9. Risk Management 12
9.1 Technical Risks 12
9.2 Project Risks 12
9.3 Issues Encountered 13
10. Conclusion 14
11. References 15

Table of Figures

Figure 1: WBS of the Betting System Project 9
Figure 2: Gantt Chart Showing part of the Project Timeline and Task Progress 10

Table of Tables

Table 1: Technical Risks 12
Table 2: Project Management Risks 13
Table 3: Issues Encountered 14

1. Introduction

1.1 Project Description

The aim of the project is to design and implement a functional prototype of a football betting system that allows users to register, log in, browse upcoming football matches, place bets, and track their betting activity. It simulates key features of real-world betting platforms, including bet selection, payout calculation and transaction management. The system incorporates multiple bet types, including Match Outcome, Double Chance, Correct Score, Over/Under Goals, Both Teams to Score, First Scorer, Total Corners, Number of Yellow Cards and Number of Red Cards.

An administrative panel is also provided, enabling management of users and matches, as well as the visualisation of financial data to support system and business analysis.

1.2 Problem Addressed

The project addresses the need to simulate a football betting system without relying on live match data. It automates match progression, removing the need for manual updates. The system also integrates betting, wallet management and bet result tracking into a single application, making it easier to test and demonstrate betting workflows in a controlled environment.

1.3 Main Functionalities

The system provides several core functionalities, including user registration and login using validated credentials. Once authenticated, users can view upcoming football matches, search for matches by team name or filter by league.

Users can also select matches and place different types of bets (one per bet type), which are added to an in-memory bet slip. They can manage and confirm their bets, while the system automatically calculates potential winnings based on total odds and the stake entered. Additionally, users can view their betting history, deposit or withdraw funds, and edit their profile information.

On the admin side, the administrator can view all users and update their status to ‘Active’, ‘Suspended’ or ‘Banned’. Once a user is banned, their status can no longer be modified. The admin can also manage matches, view match results and add new matches. Furthermore, financial data is provided through graphs showing profit and loss over the last 12 months, transaction volumes and bet outcomes. Furthermore, an AI agent is integrated to generate financial reports for administrative use.

For simulation purposes, a match simulation system is implemented to mimic live matches by automatically updating match status, bet results and bet slip status in the database while the application is running. The user interface is updated in real-time to reflect these changes. In addition, algorithms are used to calculate betting odds. For testing purposes, match results are randomly generated when

new matches are added, simplifying the prototype while maintaining core functionality, although this does not fully reflect real-world behaviour.

1.4 Project Objectives

The project aims to achieve the following objectives:

- Follow agile methodology practices to support collaboration and continuous improvement.
- Maintain effective communication and coordination within the team.
- Design a normalised database to store required information efficiently.
- Implement security features such as password hashing and transaction handling.
- Develop a match simulation system.
- Conduct thorough testing to ensure the system works correctly and is user-friendly.
- Allow admins to add new matches to support testing.
- Implementing an AI agent to assist administrative analysis and reporting.

2. Team Roles

Roles within the team were assigned based on the individual's strengths and technical skills to ensure effective contribution throughout the project. This approach allowed each member to focus on responsibilities aligned with their expertise, hence improving overall efficiency and productivity. Team accountability was maintained through regular meetings, progress updates and shared documentation.

The **Scrum Master** was responsible for coordinating and managing the overall progress of the project. This included ensuring the team remained on schedule, facilitating communication between members and assisting in resolving any issues encountered during development.

The **Secretary** was responsible for recording meeting minutes, maintaining project documentation, and keeping track of discussions and decisions made during meetings. In addition, they supported communication within the team.

The **Development Team** was responsible for implementing the system. Members demonstrated strong programming knowledge and adaptability to evolving requirements and design changes. They also showed strong collaboration skills, enabling steady progress throughout development. The team worked iteratively, delivering incremental value by implementing and refining system features in phases.

The **Testing Team** was responsible for verifying the system functionality and quality, ensuring that the application worked correctly and as intended. Member demonstrated flexibility in adapting to different tools and testing approaches. Their duties included identifying bugs and reporting issues to the development team.

3. Tools

3.1 Communication

WhatsApp was used as the main communication platform due to its convenience for quick messaging and sharing multimedia files among team members. It enabled efficient coordination and timely updates on project progress

3.2 Documentation

The following tools were used to document the project and share access to all members:

- SharePoint
- Microsoft Word Document
- Google Docs
- Microsoft Excel

3.3 Design

The following tools were used during the system design phase:

- Canva – used to create mock interfaces
- Lucidspark – used for system modelling and diagram design (e.g., ERD diagrams)

3.4 Development Environments

The system was developed using the following environments:

- Visual studio (2022/2026) – primary IDE
- JetBrains Rider – alternative IDE for macOS users
- SQL server management studio – used for database design and management

3.5 Testing Frameworks

MSTest was used for unit testing to validate system functionalities.

3.6 Version Control Management

Version control was primarily managed through structured Git commits, with priority given to the development and testing teams. This ensured that core system components were consistently integrated and maintained. Code was reviewed prior to integration into the main project to ensure adherence to structure, naming conventions and overall coding standards.

4. Meetings

This section outlines how the team maintained regular and structured communication and tracked progress throughout the project, demonstrating collaboration and Agile practices.

4.1 Frequency of Sprint Meetings

Sprint meetings were held once a week on campus to accommodate the schedule of all team members. These meetings were used to review progress, discuss completed tasks and plan upcoming work. In addition, regular updates were shared through WhatsApp to ensure continuous communication in between meetings.

4.2 Daily Stand-ups

Daily stand-ups were conducted primarily to discuss system design, features to be implemented, as well as any issues encountered during development. These discussions helped the team identify problems early, find solutions together, and maintain consistent progress throughout the project.

4.3 Recording of Meetings Notes

Meeting discussions were recorded using a mobile recording application. After each session, the recordings were reviewed and key points related to important topics were extracted by team members. These points were then organised and documented in the official shared meeting report to ensure accurate record of meetings.

4.4 Documentation of Decisions

When ideas were proposed during meetings, they were discussed among team members to evaluate their feasibility, potential benefits and time required for implementation. If a final decision could not be reached during the meeting, further research was conducted before the topic was brought up in subsequent meetings. Depending on its priority, decisions were then finalised and documented to ensure that all actions and outcomes were properly recorded and tracked.

5. Workload division

The project tasks were organized using a collaborative approach inspired by Scrum methodology. Responsibilities were distributed among team members based on their individual strengths, skills and technical competencies to ensure efficient development and high-quality output.

5.1 Task Distribution

Task allocation was based on individual expertise. Members with strong user interface design skills focused on frontend development, while members with strong programming skills handled backend logic. Database-related responsibilities were assigned to members experienced in data management. This approach ensured that each component of the system was developed by team members best suited for the task.

5.2 Progress Monitoring

Project progress was continuously monitored through regular team meetings, GitHub version control and Gantt Chart. Progress was tracked using commits, pull requests and issue updates, allowing the team to see ongoing development progress and ensure timely tasks completion.

5.3 Git Branching Strategy

A structured Git branching strategy was adopted to maintain code organisation and stability. The main branch was reserved for stable releases, while the developer branch was used for ongoing work by the development team. Each member of the development and testing teams worked on separate branches to ensure code isolation and minimise conflicts during development.

5.4 Integration and Merging

Integration of features was carried out through pull requests, followed by peer review before merging into the main branch. Merge conflicts were resolved collaboratively to ensure consistency and maintain code quality. Additionally, differences in development environments (macOS and Windows) were considered to ensure compatibility across the team.

6. Project planning

6.1 Work Breakdown Structure (WBS)

A Work Breakdown Structure (WBS), as shown in Figure 1, was used to break down the project into smaller, manageable tasks organised in a hierarchical structure. The project was broken down into five main phases: Analysis, Design, Development, Testing and Report Drafting, with each phase further broken down into sub-tasks. This approach ensured that the full project scope was covered while keeping individual tasks practical to plan, assign and manage (Siami- Irdemoosa et al., 2015). It also improved task clarity and project organisation.

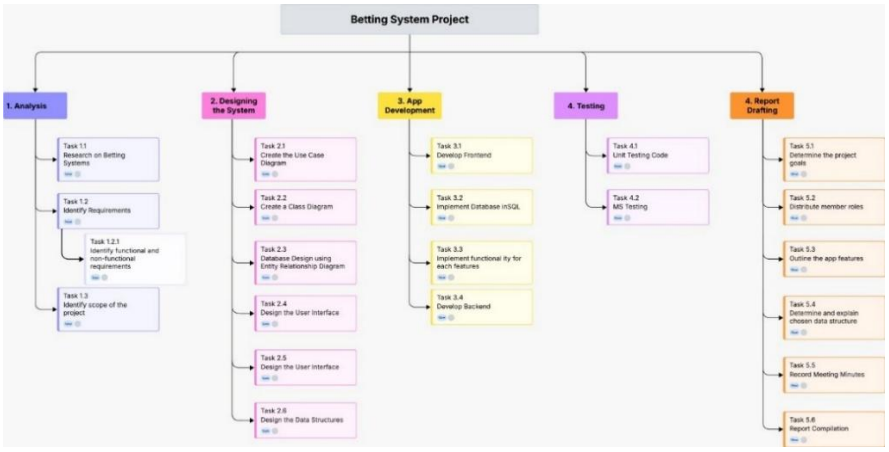
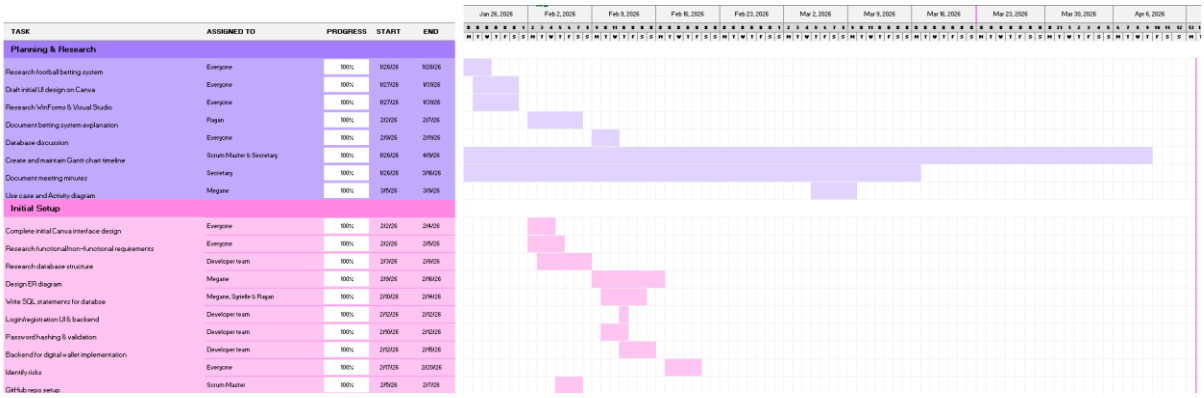


Figure 1: WBS of the Betting System Project

6.2 Gantt Chart

A Gantt-Chart was used to visually represent project tasks along with their start and end dates. Figure 2 shows an extract of the Gantt Chart developed for the project; the full version is available in the *Project Management* folder. It supported effective progress tracking and helped ensure that all tasks were completed within the planned timeframe. In addition, it facilitated the identification of task dependencies and allowed schedule adjustments in case of delays by clearly illustrating the impact on subsequent tasks and how the timeline could be modified while still meeting project deadline.



TASK	ASSIGNED TO	PROGRESS	START	END
Planning & Research				
Research football betting system	Everyone	100%	9/26/26	9/28/26
Draft initial UI design on Canva	Everyone	100%	9/27/26	9/29/26
Research WinForms & Visual Studio	Everyone	100%	9/27/26	9/29/26
Document betting system explanation	Rayan	100%	2/2/26	2/7/26
Database discussion	Everyone	100%	2/9/26	2/11/26
Create and maintain Gantt chart timeline	Scrum Master & Secretary	100%	9/26/26	4/6/26
Document meeting minutes	Secretary	100%	9/26/26	3/16/26
Use case and Activity diagram	Megane	100%	3/15/26	3/16/26
Initial Setup				
Complete initial Canva interface design	Everyone	100%	2/12/26	2/14/26
Research functional/non-functional requirements	Everyone	100%	2/12/26	2/15/26
Research database structure	Developer team	100%	2/13/26	2/16/26
Design ER diagram	Megane	100%	2/15/26	2/18/26
Write SQL statements for database	Megane, Spiele & Rayan	100%	2/16/26	2/19/26
Login/registration UI & backend	Developer team	100%	2/12/26	2/12/26
Password hashing & validation	Developer team	100%	2/16/26	2/18/26
Backend for digital wallet implementation	Developer team	100%	2/12/26	2/15/26
Identify risks	Everyone	100%	2/17/26	2/20/26
GitHub repo setup	Scrum Master	100%	2/5/26	2/7/26

Figure 2: Gantt Chart Showing Part of the Project Timeline and Task Progress

7. Project Design and Implementation

The development phase was managed using a structured and collaborative approach inspired by Agile practices, ensuring coordination between design and implementation activities. Regular team discussions and meetings were held to review progress, resolve issues and align development with project requirements.

7.1 Design Decision Coordination

Design decisions were made collaboratively during team meetings. Discussions focused on how to implement features within the system, how components would interact, the selection of appropriate data structures, and logic flow. This ensured that the system being developed remained aligned with project requirements and database design. Multiple approaches were discussed and researched to identify and follow the most efficient solution and support smooth development.

7.2 Use of Diagrams in Development

Design diagrams, including the ERD, use case and activity diagrams were used as references throughout development. They helped better understand the structure, logic and expected behaviour of the system components by clearly defining relationships, requirements and workflows.

7.3 Task Allocation in Implementation

Backed and frontend development tasks were distributed relatively equally between the developers to ensure a balanced workload and enable parallel progress. Each team member was assigned specific components of the system based on their strengths, allowing frontend and backend to be developed simultaneously.

7.4 Progress Tracking

Progress was tracked through developer check-ins, WhatsApp communication and shared documentation, where updates on completed and ongoing tasks were recorded. This was complemented by GitHub activity, including commits and pull requests, which provided a clear overview of development progress across the team.

8. Testing

The testing phase focused on ensuring that all system functionalities worked as intended, without errors, and that the developed system met the required quality standards and project requirements. The documentation for the project requirements can be found in the *Project Management* folder.

8.1 Bug Reporting Process

When new or updated functionalities were implemented and pushed to the *developer* branch by the development team, the testing team wrote and executed test cases for these features. Any bugs or incorrect system behaviour identified during testing were reported to the development team for correction, along with a clear description of the issue.

Issues were communicated during group meetings as well as via WhatsApp to ensure faster resolution. The development team addressed the reported bugs by correcting the code and pushing the updated version back to the *developer* branch. The testing team then re-tested the updated code to confirm whether issues have been resolved. This process was carried out iteratively throughout development.

In addition, the testing team assessed the overall quality of the prototype, from adherence to coding standards to the usability of the user interface.

8.2 Testing Approach

Testing was carried out by executing structured test cases for the system functionalities. Each feature was evaluated with a different set of inputs and expected outputs to ensure correct behaviour. The testing covered implemented data structures, core business-model logic, input-validation rules, service-layer behaviour in memory, and integration testing of the database access layer.
In addition, boundary testing was conducted to assess how the system behaved with invalid or extreme inputs, ensuring system stability and robustness.

8.3 Alignment with Development Sprints

Testing ran in parallel with code development to ensure continuous validation of implemented features. Each sprint included testing of completed functionalities, allowing issues to be identified and resolved early. This iterative approach ensured continuous feedback from the testing to the development teams and contributed to improved system quality.

9. Risk Management

Risk management was an important aspect of the project, allowing potential risks to be identified and appropriate mitigation strategies to be put in place. Moreover, documenting issues encountered during development provides valuable insights for reflection and can inform best practices in future projects.

9.1 Technical Risks

Table 1 outlines the key technical risks identified during system development, along with their mitigation strategies designed to reduce the likelihood of project failure.

Risk	Description	Mitigation Strategies
AI Agent Service Instability	The API may become slow or unavailable due to high request volume, rate limiting, or server-side traffic.	Error handling is implemented to ensure the system continues working if the AI service fails, along with an error message displayed to the user.
Database Integrity Issues	Multiple database operations executed consecutively could lead to data inconsistency if a failure occurs during processing.	Database transactions are implemented to ensure rollback of operations in the event of an error.
Accidental Code Overwriting	Changes made during development or testing could overwrite existing codes, leading to loss of work	A structured git branching strategy is followed with separate branches for each developer and tester to ensure code isolation and minimise risk of overwriting.

Table 1: Technical Risks

9.2 Project Risks

Table 2 outlines some risks concerning project management that could affect progress, along with their corresponding mitigation strategies to handle these risks, ensuring project remains on schedule and within the scope.

Risk	Description	Mitigation Strategies
Delay	Delays in implementing features, resolving issues or completing documentation may impact the project timeline	Tasks are prioritised and progress is monitored using a Gantt Chart and regular team meetings
Communication Issues	Ineffective communication between team members could lead to misunderstandings, or failed tasks	Regular meetings are conducted, decisions are documented, and frequent updates are shared via WhatsApp

Table 2: Project Management Risks

9.3 Issues Encountered

Table 3 outlines the main issues encountered during system development and how the team managed to resolve them.

Issue	Description	How was it faced?
Understanding Betting Logic	The implementation of betting rules (bet slips, odds generation, bet placement and bet types) was initially complex to grasp due to limited prior knowledge in this area.	The logic was broken down into smaller components and implemented incrementally alongside further research.
WinForms	Challenges were encountered when using WinForms, particularly in relation to UI responsiveness and issues with designer files breaking after modifications.	Additional research and experimentation were carried out to improve understanding and ensure stable UI behaviour.
Finding a data source	Difficulties were encountered in finding structured data (matches, leagues, teams, players) to populate the database.	An external API (*API-Football)* was used to fetch and export structured football data into CSVs for bulk insertion.
Simulating matches	Simulating matches within the app to better mimic a real betting environment while maintaining shorter durations for testing was challenging.	A periodic timer was implemented to automatically update database and UI elements while the application is running.
Implementing AI Agent	Difficult to define a meaningful role for the AI Agent due to limited available data and time constraints	Google Gemini API was integrated to implement a financial data analyst AI agent.
API Key Exposure	The Google Gemini API key was accidentally pushed to the repository, leading to the key being revoked.	The API Key value in *App.Config* was replaced with a placeholder (*YOUR_API_KEY)* to prevent exposure.

Populating Table (Bulk Insert Failure)	Errors occurred during bulk insertion, particularly due to the use of auto-generated primary keys for most tables, and inconsistent behaviour in SQL Server Management Studio during execution.	Temporary tables were used to resolve primary key conflicts, and a troubleshooting guide was added to the README file.

Table 3: Issues Encountered

10. Conclusion

Overall, the project was successfully planned, managed and delivered within the deadline using an Agile approach, which ensured effective team collaboration and adaptability throughout the development cycle. Structured planning tools such as WBS and Gantt Chart supported organisation, task allocation, progress monitoring and accountability.

Throughout the project, effective communication was maintained within the team through regular meetings, WhatsApp, GitHub version control, and shared documentation, which supported continuous coordination and development progress.

Although several challenges were encountered during development, appropriate strategies were implemented to resolve them, allowing the system to be adapted and completed within the required timeframe despite these issues and constraints.

As a result, a functional prototype of a football betting system was successfully delivered, meeting the core objectives and requirements of the project.

11. References

GeeksforGeeks (2026) *What is Gantt chart in project management* Available at: https://www.geeksforgeeks.org/business-studies/what-is-gantt-chart-in-project-management/ (Accessed: 5 March 2026).

GeeksforGeeks(2026) *Different types of risks in project development* Available at: https://www.geeksforgeeks.org/software-engineering/different-types-of-risks-in-software-project-development/ (Accessed: 7 April 2026).

Siami-Irdemoosa, E., Dindarloo, S.R. and Sharifzadeh, M. (2015) "Work breakdown structure (WBS) development for underground construction," *Automation in construction*, 58, pp. 85–94. Available at: https://doi.org/10.1016/j.autcon.2015.07.016 (Accessed: 24 March 2026)